arXiv:2406.05955v2 [cs.LG] 11 Jun 2024

Turbo Sparse: Achieving LLM SOTA Performance with Minimal Activated Parameters

Yixin Song[1], Haotong Xie[1], Zhengyan Zhang[2], Bo Wen[1], Li Ma[3], Zeyu Mi[1]✉, and Haibo Chen[1]

[1]Institute of Parallel and Distributed Systems (IPADS), Shanghai Jiao Tong University
[2]Department of Computer Science and Technology, Tsinghua University
[3]Shanghai Artificial Intelligence Laboratory

Abstract

Exploiting activation sparsity is a promising approach to significantly accelerating the inference process of large language models (LLMs) without compromising performance. However, activation sparsity is determined by activation functions, and commonly used ones like SwiGLU and GeGLU exhibit limited sparsity. Simply replacing these functions with ReLU fails to achieve sufficient sparsity. Moreover, inadequate training data can further increase the risk of performance degradation. To address these challenges, we propose a novel dReLU function, which is designed to improve LLM activation sparsity, along with a high-quality training data mixture ratio to facilitate effective sparsification. Additionally, we leverage sparse activation patterns within the Feed-Forward Network (FFN) experts of Mixture-of-Experts (MoE) models to further boost efficiency. By applying our neuron sparsification method to the Mistral and Mixtral models, only 2.5 billion and 4.3 billion parameters are activated per inference iteration, respectively, while achieving even more powerful model performance. Evaluation results demonstrate that this sparsity achieves a 2-5× decoding speedup. Remarkably, on mobile phones, our TurboSparse-Mixtral-47B achieves an inference speed of 11 tokens per second. Our models are available at `https://huggingface.co/PowerInfer`.

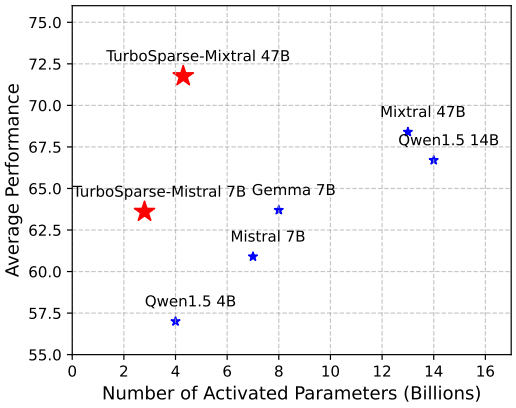


Figure 1: Comparison on the Open LLM Leaderboard shows that our dReLU-based sparsified models, particularly TurboSparse-Mixtral-47B, consistently outperform similar models.

✉Corresponding author: Zeyu Mi (`yzmizeyu@sjtu.edu.cn`).

Preprint. Under review.

1 Introduction

Large Language Models (LLMs) have achieved remarkable results, demonstrating emergent natural language abilities as the number of model parameters scales [9, 67]. These models have pushed the state-of-the-art performance across a wide range of downstream applications, such as QA and coding. However, most LLMs, such as Llama [60], Mistral [24], and Gemma [58], utilize all of their parameters during inference. These are known as dense models. The escalating demand for computational resources by dense models has become a significant barrier to the development of powerful and accessible AI, given the substantial costs involved.

To address the efficiency issues inherent in dense models, conditional computation [7, 6] has emerged as a crucial approach, which refers to activating part of the neurons in a network. There are two primary methods to achieve conditional computation. Mixture-of-Experts (MoE) [17, 31] is the first promising method, which introduces conditional computation by manually setting constraints on the model architecture prior to training, such as determining the number of experts to activate. This technique selectively activates specific parts of the model in response to particular inputs through a process known as expert routing, resulting in significant efficiency improvements. For instance, Switch Transformer [17] has scaled the model to the trillion-parameter level without increasing computational FLOPs significantly. Another promising method is utilizing the natural emergence of sparse activation due to the ReLU activation function [33], which naturally outputs zero elements that have no contribution in computation results. This activation sparsity presents a significant opportunity for efficient inference. Deja Vu [36] utilizes that sparsity exists in dense models due to ReLU to achieve $2\times$ speedups. PowerInfer [56] achieving up to $11\times$ speedups for deploying larger LLMs in a single consumer-grade GPU setting.

Recent LLMs typically prefer activation functions such as GELU [23] and Swish [50]. However, these functions do not significantly promote activation sparsity and are challenging to accelerate with conditional computation. To address this, ReLUfication [42], an existing state-of-the-art method, replaces the original activation function with ReLU and continues with pretraining. Despite its potential, this approach often struggles to achieve the desired levels of activation sparsity and may risk performance degradation [30, 59].

We argue that the failure of existing ReLUfication methods can be attributed to two main reasons. First, simply substituting SwiGLU with ReGLU is inefficient, as it only increases sparsity from 40% to around 70%. It suggests that a deeper investigation into the model architecture is necessary to achieve higher levels of sparsity. Second, the limited diversity of pretraining data and the insufficient number of training tokens in current approaches lead to incomplete capability recovery [42, 30]. As a result, expanding the diversity of pretraining datasets and increasing the number of training tokens are critical steps towards enhancing model performance.

To address these challenges, we first conduct a comprehensive analysis of the existing ReLUfication approach and identify that its shortcomings stem from the negative activations in the GLU component. Therefore, we propose an efficient activation function named dReLU. We apply dReLU in the pretraining of small-scale LLMs, alongside SwiGLU, and our findings indicate that LLMs using dReLU match the performance of those using SwiGLU, while also achieving close to 90% sparsity. Additionally, we collect a diverse range of pretraining corpora from the open-source community, including web, code, and mathematical datasets, to enhance the effectiveness of ReLUfication.

Meanwhile, we also conduct a sparsity analysis on MoE-based LLMs. Interestingly, we observe that the feed-forward networks (FFNs) within the experts remain sparsely activated, similar to the behavior exhibited by dense LLMs. This phenomenon suggests an opportunity to further accelerate inference speed by combining MoE techniques with ReLU-based sparse activation.

To validate the effectiveness of our proposed method, we implemented it on the Mistral-7B and Mixtral-47B models, converting them to TurboSparse-Mistral-47B and TurboSparse-Mixtral-47B, respectively. Extensive experiments across a wide range of downstream tasks demonstrate (Figure 1) that our enhanced models not only meet but often surpass the performance of their original counterparts.

Remarkably, in the TurboSparse-Mistral-7B model, we increase the average sparsity of the FFN to 90% while enhancing model capabilities. In MoE models, we further improve the sparsity in the TurboSparse-Mixtral-47B, originally introduced due to expert routing, from 75% to 97% by

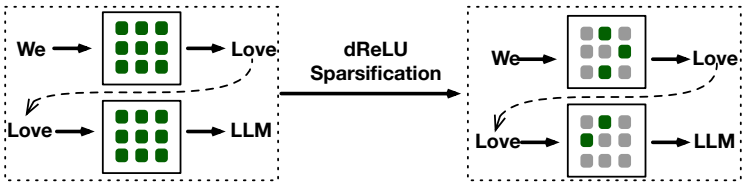


Figure 2: Example of dReLU sparsification. The left figure illustrates the original dense activation where every input activates all neurons, while the right is our sparsified LLMs, where each input activates only a small subset of neurons.

incorporating sparse neuron activations. This substantial increase in sparsity significantly reduces FLOPs during the inference process.

Finally, we integrate our two new models with PowerInfer to evaluate the inference speed. Performance evaluation reveals that our models deliver an average 2.83× generation speedup.

The key contributions of this paper include:

- **Efficient dReLU activation function:** Our method utilizes fewer than 150B tokens, representing less than 1% of the typical pretraining tokens (commonly 15T tokens [11]).
- **Sparse activated models:** We will release our sparsely-activated TurboSparse-Mistral-7B and TurboSparse-Mixtral-47B models. Both models demonstrate better performance compared to the original versions.
- **Practical inference speedup:** Evaluation shows that with our models, we can achieve a 2-5× speedup. Notably, we can achieve up to 10 tokens/s even without a GPU on TurboSparse-Mixtral-47B.

2 Related Work and Background

Efficient Inference of LLMs. Efficient LLM inference poses challenges that necessitate a synergistic combination of algorithmic and systemic approaches. From an algorithmic standpoint, researchers have explored various methods to reduce computation and memory overheads, including compressing models [40, 63, 18, 34, 61], modifying model structures [3, 21], and speculative decoding methods [32, 12, 10]. On the systemic front, there are efforts that effectively integrate the features of downstream hardware and upper-level models to maximize the efficiency of computation and memory utilization [4, 49, 16, 64], leading to the development of more efficient frameworks like vLLM [29].

Sparse activation, in particular, has emerged as a research area that demands an even tighter integration of algorithmic and systemic approaches. The selection of activation functions and the construction of activation predictors are algorithmic problems, while fully exploiting the sparse activation of LLMs on specific hardware is a systemic challenge. By leveraging sparse activation, researchers have achieved promising results in building efficient LLM inference systems [36, 56].

Mixture-of-Experts (MoE). MoE techniques induce effective sparsity in LLMs by determining which subset of subnetworks (referred to as "experts") to activate during the inference pass, often through a trained "router" subnetwork. This approach allows the model to enhance its capacity without escalating the computational expenses [31, 53].

Intrinsic Activation Sparsity. Intrinsic activation sparsity is known to be present in LLMs that utilize ReLU family nonlinearities in their MLP blocks [68, 33]. This phenomenon has been explored to accelerate inference speed and reduce memory usage [56, 36, 37]. With this phenomenon, each neuron can be viewed as an expert to reduce the computation overhead.

Gated-MLP Blocks. We now delve into the components of LLMs that our study aims to analyze: the Gated-MLP blocks, which are commonly used. A Gated-MLP block consists of three fully

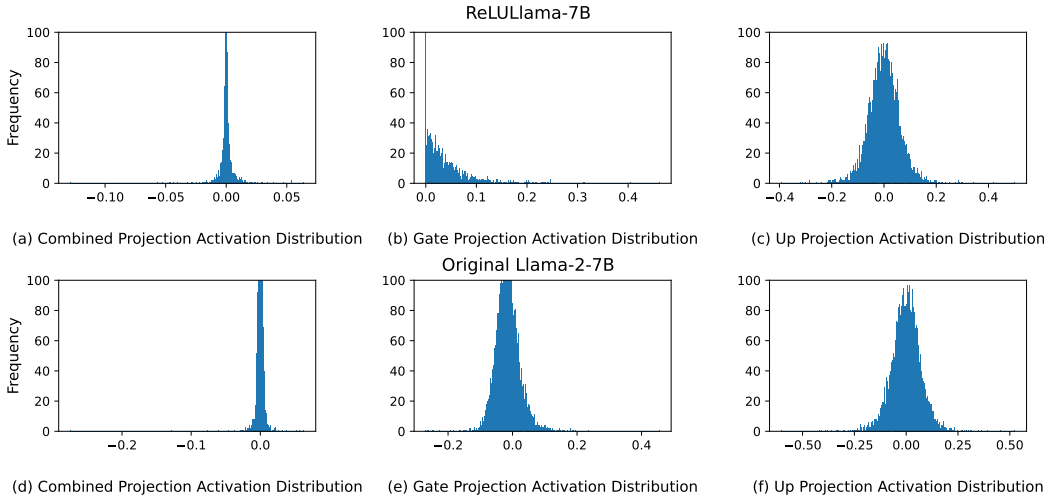
connected layers and performs the following computation:

$$
\begin{aligned}
\text{Gate}(x) &:= F_{act}(xW_{\text{gate}}) \\
\text{Up}(x) &:= xW_{\text{up}} \\
\text{Combined}(x) &:= \text{Gate}(x) * \text{Up}(x) \\
\text{Gated-MLP}(x) &:= \text{Combined}(x)W_{\text{down}}
\end{aligned}
\tag{1}
$$

where F_{act} represents different activation functions like ReLU [2], SiLU [38].

3 Analysis

3.1 Limitations about Existing ReLUfication



(a) Combined Projection Activation Distribution (b) Gate Projection Activation Distribution (c) Up Projection Activation Distribution

(d) Combined Projection Activation Distribution (e) Gate Projection Activation Distribution (f) Up Projection Activation Distribution

Figure 3: Post-activation distribution of ReLULlama and Llama-2-7B in layer 0.

Table 1: Model Sparisty compared ReLULlama with Llama-2-7B

Model	Sparisty
Llama-2-7B	40%
ReLULlama-7B	67%
ShiftedReLULlama-7B	71%

We first evaluate the sparsity of ReLULlama-7B [59] and the original Llama-2-7B [60], as shown in Table 1. The results reveal that existing ReLUfication methods can only improve the sparsity from 40% to 67%, indicating their limited effectiveness in significantly enhancing model sparsity.

To investigate the underlying reasons for this limitation, we profile the activation distribution of the gate and up projection components separately in ReLULlama-7B and Llama-2-7B, as illustrated in Figure 3. The figure shows that after ReLUfication, the combined activation becomes more concentrated around 0, with the sparsity increasing to 67%. This can be attributed to the ReLU activation function applied after the gate weight, which masks all negative activations to zero.

To further push the sparsity, shifted-ReLU [42] has been proposed, which adjusts the threshold of ReLU function to mask out more activations in the gate projection. However, the improvements brought by this method are limited. Another line of work is to adopt progressive sparsity regularization to the intermediate output to introduce more zero activation output [55]. However, this method carries the risk of performance degradation.

Existing ReLUfication methods primarily focus on modifying the gate component. Different from previous work, we find that existing ReLUfication doesn't alter the activation distribution of the up projection component, as shown in Figure 3(c) and (f). According to the definition of Gated-MLP (Equation 1), the gate and up projection components jointly influence the sparsity of neuron activations in parallel. However, a significant number of activation values in the up projection component remain less than 0. This suggests that masking the outputs of the up and gate matrices that are less than 0 as

inactive could introduce stronger sparsity without sacrificing non-linear capabilities. This observation motivates us to explore the possibility of further enhancing model sparsity by modifying the up projection.

3.2 dReLU

We introduce a new activation function, named dReLU (Equation 2), where ReLU is applied after both the up- and gate-projection[1].

$$\text{Combined}_{\text{dReLU}}(x) := \max(0, xW_{\text{gate}}) * \max(0, xW_{\text{up}}) \tag{2}$$

To demonstrate the effectiveness and performance of dReLU, we conducted an experiment comparing 300M-parameter decoder-only architecture models using dReLU and SwiGLU, both pretrained under the fineweb dataset [47] for 5B tokens. Refer to Appendix A.1 for the detailed model architecture hyperparameters. The evaluation result is shown in Table 2.

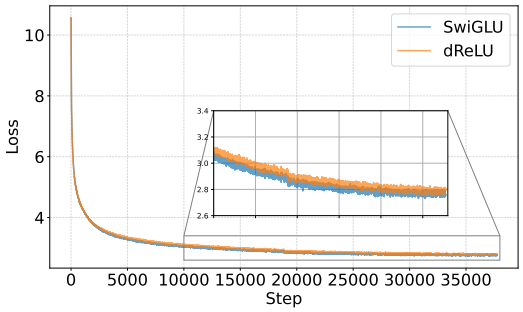


Figure 4: Training loss of small models with different activation functions.

Activation	Training Loss	Validation PPL
dReLU	3.154	28.45
SwiGLU	3.146	28.48

Table 2: Validation and training loss on different activations.

Our findings reveal models employing the dReLU structure exhibit similar convergence compared to those using the SwiGLU structure. Notably, we evaluate the perplexity of both models on Wikitext-2 [39]. DReLU-based models show slightly better performance on WikiText-2 [39].

Figure 4 illustrates the loss curves during training, demonstrating that models with the dReLU activation function achieve similar convergence ability compared to their SwiGLU counterparts. To further validate this observation, we evaluate the perplexity of these models on the Wikitext2 dataset. As shown in Table 2. Notably, although SwiGLU-based model has lower training loss, dReLU based model has lower validation perplexity. These results provide strong evidence that adopting the dReLU structure does not compromise model performance. We evaluate on more downstream tasks in Appendix A.1.

Another question we need to address is the dReLU-based model's sparsity. To investigate the sparsity of the dReLU-based model, we propose a methodology for measuring and evaluating a model's performance under different sparsity levels. Our approach involves selecting the top-$k\%$ of values activated by dReLU or other activation functions based on their absolute magnitude, as described in Equations 3 and 4.

$$\text{Mask}(x) := \text{Top}_k(|\text{Combined}(x)|) \tag{3}$$

$$\text{Gated-MLP}(x) := (\text{Combined}(x) * \text{Mask}(x))W_{\text{down}} \tag{4}$$

where W_{down} represents the down-projection matrix. By varying the value of k, we can control the sparsity level of the model. To assess the impact of sparsity on model performance, we evaluate the dReLU-based model on a range of downstream tasks at different sparsity levels. This allows us to determine the optimal sparsity-performance trade-off for the dReLU activation function. Table 3 presents the perplexity of the dReLU-based and SwiGLU-based models on WikiText-2 across various sparsity levels. The results demonstrate that the dReLU activation function enables high sparsity without significant degradation in performance, maintaining competitive performance even at 90% sparsity. In contrast, the SwiGLU-based model experiences a severe increase in perplexity as sparsity reaches 80%.

[1]We omit the bias in both the up- and gate-projection to match the form of Equation 1.

Top-k%	0	50%	80%	85%	90%
dReLU	28.45	28.45	28.45	28.65	29.19
SwiGLU	28.48	28.62	36.28	48.55	112.36

Table 3: WikiText-2 perplexity on different sparsity on different models.

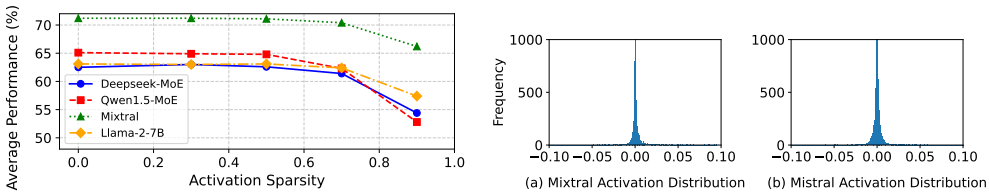
4 Are Neurons in Expert still Sparsely Activated?

Previous work has shown that dense LLMs with different activation functions (ReLU, SwiGLU, etc.) exhibit the property of sparse activation [69, 36, 30]. However, the analysis is limited to dense models. Despite the intuitive assumption that partitioning FFNs into different experts within an MoE model would result in denser activations within each expert, it remains unclear whether this sparsity phenomenon persists in MoE models. In this section, we select representative MoE models and commonly used downstream tasks to investigate whether this sparsity phenomenon still exists in MoE models. We utilize the same method in 3 to control the sparsity in each expert.

Models. We select Deepseek-MoE [15], Qwen1.5-MoE [5] and Mixtral [25] as the models for our experiments. We also add Llama-2-7B as for comparison.

We first study the performance with regard to the sparsity ratio, as shown in Figure 5 (a)[2]. Specifically, the performance only drops by about 1%-2% when the sparsity ratio is 0.5. This trend suggests that MoE models exhibit similar sparsity compared to dense models.

Further, we profile the activation patterns of Mistral and Mixtral, a pair of popular dense LLM and MoE LLM, as shown in Figure 5 (b). We find that both LLMs show a similar pattern where activations are concentrated around 0, which is consistent with previous analysis of dense LLMs. The sparsity in experts also implies that every neuron in the same expert has different functionality. This finding applies to all layers and experts, as detailed in Appendix A.2. We report this interesting observation and leave further analysis for future work.



(a) **(b)**

Figure 5: (a) Performance of MoE models with regard to activation sparsity. The impact of activation sparsity on the performance is negligible until the sparsity ratio is larger than 0.5. (b) Activation distribution of Mixtral and Mistral.

Inspired by our discoveries in MoE models, we are convinced that ReLUfication can be extended to MoE models and is not restricted to dense models. As the proportion of FFN weights in MoE models increases, the FLOP reduction achieved through ReLUfication will be even more pronounced.

5 dReLU Sparsification

In the previous section, we have demonstrated that dReLU can be a better choice for ReLUfication. The main question now is whether dReLU based ReLUfication can recover the original model's performance while achieving higher sparsity. The following sections will discuss the experiments that aimed at answering this question.

Experimental setup. We consider two representative models: Mistral-7B and Mixtral-47B. We substitute the original SwiGLU based FFN with dReLU based FFN and then continue pretraining.

[2]Each performance score is reported as an average of Winogrande [51], TruthfulQA [35], PIQA [8], LAMBADA [45], ARC-Easy [13], and ARC-Challenge [13].

Pretraining datasets. Due to the ReLUfication process, the restoration of model capability is closely related to the corpus used for recovery training. We collected as much corpus as possible from the open-source community for training, such as Wanjuan-CC [48], open-web-math [46], peS2o [54], Pile [19], The Stack [28], GitHub Code [1] and so on. The detailed mixture ratio is as shown in the following table 4:

Category	Dataset	Percentage
Academic	Pile-Arxiv	
	Pile-PubMed	2.69%
	Pile-Philpapers	
	Dolma_peS2o	7.83%
Web	Wanjuan-CC	73.55%
	RedPajama-Wiki	0.65%
	Pile-OpenWebtext2	0.44%
Books	RedPajama-books	6.54%
	Pile-PG19	0.37%
Math	Open-web-math	1.46%
	Proof-pile-2	0.76%
	algebraic-stack	0.54%
Code	Starcoder-Java	0.77%
	Starcoder-C#	0.74%
	Starcoder-Typescript	0.52%
	Starcoder-remaining	1.73%
	GitHub-Code	1.41%

Table 4: Detailed data mixture

SFT datasets. After pretraining, we utilize the high-quality SFT datasets to further improve our model's performance, including orca-math-word-problems [43], bagel [27].

Hyper-parameters. The hyperparameters for our ReLUfication are based on empirical results from previous works [69]. We utilize the llm-foundry framework for training [44] and employ FSDP parallelism.

Our models are trained using the AdamW optimizer [38] with the following hyper-parameters: $\beta_1 = 0.9$ and $\beta_2 = 0.95$. We adopt a cosine learning rate schedule and use the default values for weight decay and gradient clipping (see Table 5 for more details). In total, we pretrain our models on 150B tokens.

Table 5: Details of training hyper-parameters.

Sequence Length	Batch Size	Learning Rate	Warmup Steps	Hardware
4,096	2,048	$5e^{-5} \rightarrow 5e^{-6}$	1000	64 A800-80G GPUs

6 Experiments Results

6.1 Downstream Tasks Performance

We measure our sparsified models' performance on tasks included in OpenLLM Leaderboard which include 25-shot Arc-Challenge [13], 10-shot Hellaswag [65], 5-shot MMLU [22], 0-shot TruthfulQA [35], 5-shot Winogrande [51] and 8-shot GSM8K [14]. In addition, we also follow Llama 2's evaluation task included commonsense reasoning tasks. We report the average of PIQA [8], SCIQ [26], ARC easy [13], OpenBookQA [41]. We compare our models to several external open-source LLMs, including Gemma-2B [58], Mistral-7B [24] and Mixtral-47B [25].

	Gemma -2B	Mistral -7B	TurboSparse -Mistral-7B	Mixtral -47B	TurboSparse -Mixtral-47B
# Total Params	2B	7B	7B	47B	47B
# Activate Params	2B	7B	2.5B	13B	**4.3B**
ARC-challenge	48.55	61.43	62.2	68.09	67.49
Hellaswag	71.02	83.32	82.17	86.62	85.22
MMLU	40.05	62.65	63.89	70.53	70.48
TruthfulQA	34.38	44.06	46.64	48.59	56.64
WinoGrande	66.06	79.24	76.16	83.35	82.24
GSM8k	18.72	40.17	50.84	58.91	68.50
CommonSense	63.4	75.8	76.2	78.07	78.52
OpenLLM Leaderboard Avg.	46.46	61.57	63.65	69.34	71.76

Table 6: Downstream benchmarks results from four different models.

Table 6 shows the results from different models. TurboSparse-Mistral-7B outperforms Gemma-2B by far, while only activating 3B parameters. TurboSparse-Mixtral-47B outperforms the original Mixtral-47B with only 4.5B parameters activated. The results demonstrate that LLMs with ReLU based intrinsic activation sparsity can keep the same or better performance while hold the significant FLOPs reduction.

6.2 Sparsity of Sparsified Models

In this subsection, we report our models' sparsity. We first profile the proportion of zero-valued activations for every layer with a general dataset(fineweb), as shown in Figure 6. By considering activations with a value of zero, we find that for TurboSparse-Mistral-7B, on average, has 90% of the neurons inactive in each layer. For TurboSparse-Mixtral-47B, this percentage is slightly lower at 85% on average for each expert FFN. Originally, Mixtral-47B would activate 2 out of 8 experts in each layer, introducing 75% sparsity, meaning only 25% of FLOPs needed to be computed. Furthermore, after ReLUfication, each expert will only activate 15% of neurons. Combining these, in inference, only 3% of parameters in each MoE layer will be activated.

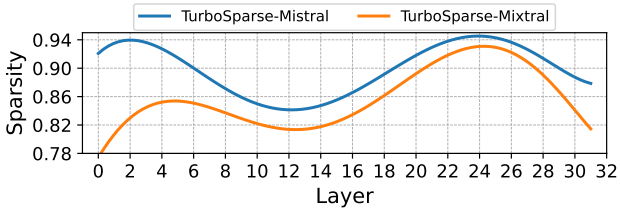


Figure 6: Sparsity of TurboSparse-Mistral-7B and TurboSparse-Mixtral-47B of different layers.

7 Practical Inference Speedup Evaluation

In this section, we evaluate the practical acceleration in model generation achieved. During the SFT phase, we incorporate a predictor module for each FFN block. Notably, for the TurboSparse-Mixtral-47B, we train predictors for each expert. When an expert is routed, the neuron-level predictor identifies which neurons will be activated, enabling neuron-level sparse computation.

We integrate our two models with PowerInfer, which is a state-of-the-art sparsely-activated framework to evaluate the actual generation speed.

Table 7: Decoding Speed with CPU only (tokens/s)

Setting	Model	PowerInfer	llama.cpp	Speedup
PC-2080Ti	Mistral-7B-FP16	9.94	4.78	2.08×
PC-2080Ti	Mixtral-47B-INT4	11.98	4.26	2.81×
PC-Laptop	Mistral-7B-FP16	8.71	4.13	2.11×
PC-Laptop	Mixtral-47B-INT4	16.1	6.91	2.32×

Table 8: Decoding Speed with CPU/GPU hybrid computing (tokens/s)

Setting	Model	PowerInfer	llama.cpp	Speedup
PC-2080Ti	Mistral-7B-FP16	35.5	7.64	4.64×
PC-2080Ti	Mixtral-47B-INT4	22.24	6.63	3.35×
PC-Laptop	Mixtral-47B-INT4	33.12	13.1	2.52×

7.1 Experiments Setting

Baselines. We take llama.cpp [20] as our baselines for comparison. llama.cpp is the most representative inference framework.

Models. For PowerInfer and PowerInfer-2 [62], we deployed our sparsified models, while for llama.cpp, we employed the original models for speed comparison.

Hardware Configurations. All experiments were conducted on three distinct configurations:

- **PC-Laptop**: Intel i9-14900HX processor, 32GB host memory (67.2 GB/s bandwidth), an NVIDIA RTX 4090 GPU (16GB), and PCIe 4.0 interface (64GB/s bandwidth).
- **PC-2080Ti**: Intel i7-12700K processor (eight 4.9GHz cores), 64GB host memory (38.4 GB/s bandwidth), an NVIDIA RTX 2080Ti GPU (11GB), and PCIe 3.0 interface (32GB/s bandwidth).
- **OnePlus-12:** Equipped with a Snapdragon 8 Gen 3 SoC, 24 GB DRAM, and UFS 4.0 storage.

7.2 Pure CPU Inference

In this subsection, we focus on utilizing only the CPU for inference in our models. Due to limitations in DRAM, our evaluations are constrained to CPU performance. Table 7 presents the decoding speed results achieved with CPU-only processing for different models and settings.

The table provides a comparison of decoding speeds (in tokens per second) for various models under different settings using CPU-only inference. Overall, our ReLUfied models can achieve 2.08-2.28× speedup over the original model.

7.3 Hybrid GPU-CPU Inference

In this subsection, we shift our focus to evaluating our models in a hybrid GPU-CPU computing environment, considering that most PCs are equipped with consumer-grade GPUs. Table 8 presents the decoding speed results achieved with hybrid GPU-CPU computing for different models and settings.

The table below provides a comparison of decoding speeds (in tokens per second) for various models under different settings using a combination of GPU and CPU for inference. Overall, our models demonstrate significant speedups ranging from 2.52 to 4.64× compared to the baseline llama.cpp.

7.4 Deploy LLMs on mobile phones

We also serve TurboSparse-Mixtral-47B by using PowerInfer-2 that supports LLM inference on mobile phones. PowerInfer-2 leverages the sparse activation feature during LLM inference and

Table 9: Decoding Speed on Mobile Phones (tokens/s)

Setting	Model	PowerInfer-2	llama.cpp	Speedup
OnePlus-12	Mixtral-47B-INT4	11.1	0.5	22.2×

introduces a computational engine on heterogeneous XPUs. It can perform high-speed inference even when the model parameters exceed DRAM capacity. As shown in Table 9, PowerInfer-2 achieves a 22.2× speedup using TurboSparse-Mixtral-47B inference compared to llama.cpp with the original Mixtral-47B. This significant performance gain is primarily because PowerInfer-2 can fully exploit the extremely high sparsity that TurboSparse demonstrates during inference.

8 Conclusion

We propose a novel dReLU-based sparsification method that increases model sparsity to 90% while maintaining performance, achieving a 2-5× speedup in inference. This method significantly reduces resource requirements for deploying large models, making them more environmentally friendly and accessible. This breakthrough is expected to accelerate the development of natural language processing technologies, benefiting a wider range of users. We believe that the dReLU-based sparsification method will be crucial for efficient, high-performing, and widely accessible LLMs.

References

[1] Github code dataset. `https://huggingface.co/datasets/codeparrot/github-code`, 2022.

[2] Abien Fred Agarap. Deep learning using rectified linear units (relu), 2019.

[3] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of EMNLP*, pages 4895–4901, 2023.

[4] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, and Yuxiong He. Deepspeed- inference: Enabling efficient inference of transformer models at unprecedented scale. In *Proceedings of SC*, 2022.

[5] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, Keming Lu, Jianxin Ma, Rui Men, Xingzhang Ren, Xuancheng Ren, Chuanqi Tan, Sinan Tan, Jianhong Tu, Peng Wang, Shijie Wang, Wei Wang, Shengguang Wu, Benfeng Xu, Jin Xu, An Yang, Hao Yang, Jian Yang, Shusheng Yang, Yang Yao, Bowen Yu, Hongyi Yuan, Zheng Yuan, Jianwei Zhang, Xingxuan Zhang, Yichang Zhang, Zhenru Zhang, Chang Zhou, Jingren Zhou, Xiaohuan Zhou, and Tianhang Zhu. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.

[6] Emmanuel Bengio, Pierre-Luc Bacon, Joelle Pineau, and Doina Precup. Conditional computation in neural networks for faster models. *arXiv preprint arXiv:1511.06297*, 2015.

[7] Yoshua Bengio. Deep learning of representations: Looking forward. In *International conference on statistical language and speech processing*, pages 1–37. Springer, 2013.

[8] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. Piqa: Reasoning about physical commonsense in natural language. In *Thirty-Fourth AAAI Conference on Artificial Intelligence*, 2020.

[9] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[10] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv: 2401.10774*, 2024.

[11] Zheng Cai, Maosong Cao, Haojiong Chen, Kai Chen, Keyu Chen, Xin Chen, Xun Chen, Zehui Chen, Zhi Chen, Pei Chu, et al. Internlm2 technical report. *arXiv preprint arXiv:2403.17297*, 2024.

[12] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

[13] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv:1803.05457v1*, 2018.

[14] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[15] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. *CoRR*, abs/2401.06066, 2024.

[16] Jiarui Fang, Yang Yu, Chengduo Zhao, and Jie Zhou. Turbotransformers: an efficient GPU serving system for transformer models. In *Proceedings of PPoPP*, 2021.

[17] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[18] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate quantization for generative pre-trained transformers. In *The Eleventh International Conference on Learning Representations*, 2023.

[19] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, et al. The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020.

[20] Georgi Gerganov. ggerganov/llama.cpp: Port of facebook's llama model in c/c++. `https://github.com/ggerganov/llama.cpp`, 2023.

[21] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

[22] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[23] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.

[24] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7b, 2023.

[25] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mixtral of experts, 2024.

[26] Matt Gardner Johannes Welbl, Nelson F. Liu. Crowdsourcing multiple choice science questions. 2017.

[27] jondurbin. A bagel, with everything, 2024.

[28] Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, Dzmitry Bahdanau, Leandro von Werra, and Harm de Vries. The stack: 3 tb of permissively licensed source code. *Preprint*, 2022.

[29] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of SOSP*, pages 611–626, 2023.

[30] Je-Yong Lee, Donghyun Lee, Genghan Zhang, Mo Tiwari, and Azalia Mirhoseini. Cats: Contextually-aware thresholding for sparsity in large language models, 2024.

[31] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.

[32] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of ICML*, 2023.

[33] Zonglin Li, Chong You, Srinadh Bhojanapalli, Daliang Li, Ankit Singh Rawat, Sashank J Reddi, Ke Ye, Felix Chern, Felix Yu, Ruiqi Guo, et al. The lazy neuron phenomenon: On emergence of activation sparsity in transformers. *arXiv preprint arXiv:2210.06313*, 2022.

[34] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, and Song Han. AWQ: activation-aware weight quantization for LLM compression and acceleration. *arXiv preprint arXiv:2306.00978*, 2023.

[35] Stephanie Lin, Jacob Hilton, and Owain Evans. Truthfulqa: Measuring how models mimic human falsehoods, 2021.

[36] Zichang Liu, Jue Wang, Tri Dao, Tianyi Zhou, Binhang Yuan, Zhao Song, Anshumali Shrivastava, Ce Zhang, Yuandong Tian, Christopher Re, and Beidi Chen. Deja vu: Contextual sparsity for efficient LLMs at inference time. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 22137–22176. PMLR, 23–29 Jul 2023.

[37] Zirui Liu, Qingquan Song, Qiang Charles Xiao, Sathiya Keerthi Selvaraj, Rahul Mazumder, Aman Gupta, and Xia Hu. Ffsplit: Split feed-forward network for optimizing accuracy-efficiency trade-off in language model inference. *arXiv preprint arXiv:2401.04044*, 2024.

[38] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

[39] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models, 2016.

[40] Paul Michel, Omer Levy, and Graham Neubig. Are sixteen heads really better than one? In *Proceedings of NeurIPS*, 2019.

[41] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *EMNLP*, 2018.

[42] Iman Mirzadeh, Keivan Alizadeh, Sachin Mehta, Carlo C Del Mundo, Oncel Tuzel, Golnoosh Samei, Mohammad Rastegari, and Mehrdad Farajtabar. Relu strikes back: Exploiting activation sparsity in large language models. *arXiv preprint arXiv:2310.04564*, 2023.

[43] Arindam Mitra, Hamed Khanpour, Corby Rosset, and Ahmed Awadallah. Orca-math: Unlocking the potential of slms in grade school math, 2024.

[44] Mosaicml. Llm foundry. `https://github.com/mosaicml/llm-foundry`, 2023.

[45] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Quan Ngoc Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The lambada dataset, Aug 2016.

[46] Keiran Paster, Marco Dos Santos, Zhangir Azerbayev, and Jimmy Ba. Openwebmath: An open dataset of high-quality mathematical web text, 2023.

[47] Guilherme Penedo, Hynek Kydlíček, Leandro von Werra, and Thomas Wolf. Fineweb. 2024.

[48] Jiantao Qiu, Haijun Lv, Zhenjiang Jin, Rui Wang, Wenchang Ning, Jia Yu, ChaoBin Zhang, Zhenxiang Li, Pei Chu, Yuan Qu, Jin Shi, Lindong Lu, Runyu Peng, Zhiyuan Zeng, Huanze Tang, Zhikai Lei, Jiawei Hong, Keyu Chen, Zhaoye Fei, Ruiliang Xu, Wei Li, Zhongying Tu, Hang Yan, and Conghui He. Wanjuan-cc: A safe and high-quality open-sourced english webtext dataset, 2024.

[49] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. Deepspeed-moe: Advancing mixture-of-experts inference and training to power next-generation AI scale. In *Proceedings of ICML*, 2022.

[50] Prajit Ramachandran, Barret Zoph, and Quoc V Le. Searching for activation functions. *arXiv preprint arXiv:1710.05941*, 2017.

[51] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106, 2021.

[52] Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

[53] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.

[54] Luca Soldaini and Kyle Lo. peS2o (Pretraining Efficiently on S2ORC) Dataset. Technical report, Allen Institute for AI, 2023. ODC-By, `https://github.com/allenai/pes2o`.

[55] Chenyang Song, Xu Han, Zhengyan Zhang, Shengding Hu, Xiyu Shi, Kuai Li, Chen Chen, Zhiyuan Liu, Guangli Li, Tao Yang, et al. Prosparse: Introducing and enhancing intrinsic activation sparsity within large language models. *arXiv preprint arXiv:2402.13516*, 2024.

[56] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. Powerinfer: Fast large language model serving with a consumer-grade gpu. *arXiv preprint arXiv:2312.12456*, 2023.

[57] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[58] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, et al. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*, 2024.

[59] SpaseLLM Team. Sparse large language models with relu activation, 2023.

[60] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[61] Guangxuan Xiao, Ji Lin, Mickaël Seznec, Hao Wu, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. In *Proceedings of ICML*, 2023.

[62] Zhenliang Xue, Yixin Song, Zeyu Mi, Le Chen, Yubin Xia, and Haibo Chen. Powerinfer-2: Fast large language model inference on a smartphone, 2024.

[63] Zhewei Yao, Reza Yazdani Aminabadi, Minjia Zhang, Xiaoxia Wu, Conglong Li, and Yuxiong He. Zeroquant: Efficient and affordable post-training quantization for large-scale transformers. In *Proceedings of NeurIPS*, 2022.

[64] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *Proceedings of OSDI*, 2022.

[65] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.

[66] Biao Zhang and Rico Sennrich. Root Mean Square Layer Normalization. In *Advances in Neural Information Processing Systems 32*, Vancouver, Canada, 2019.

[67] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. Opt: Open pre-trained transformer language models, 2022.

[68] Zhengyan Zhang, Yankai Lin, Zhiyuan Liu, Peng Li, Maosong Sun, and Jie Zhou. Moefication: Transformer feed-forward layers are mixtures of experts. *arXiv preprint arXiv:2110.01786*, 2021.

[69] Zhengyan Zhang, Yixin Song, Guanghui Yu, Xu Han, Yankai Lin, Chaojun Xiao, Chenyang Song, Zhiyuan Liu, Zeyu Mi, and Maosong Sun. Relu ^{2} wins: Discovering efficient activation functions for sparse llms. *arXiv preprint arXiv:2402.03804*, 2024.

[70] Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyan Luo, and Yongqiang Ma. Llamafactory: Unified efficient fine-tuning of 100+ language models. *arXiv preprint arXiv:2403.13372*, 2024.

A Appendix / supplemental material

A.1 Training Details of 300M models

In this subsection, we will introduce the details of training the 300M model, including the model architecture, types of data used, and hyperparameters. The evaluation results of the final 300M models are shown in Table 10.

Table 10: Accuracy (%) of models on evaluation datasets with average.

	ARC:E	ARC:C	PIQA	Winogrande	BoolQ	HellaSwag	LAMBADA	Average
SwiGLU	40.03	22.95	62.68	52.72	60.92	31.63	24.34	42.18
DReLU	40.07	22.44	63.82	52.33	61.50	31.08	24.35	42.23

A.1.1 Architecture

We adopt a similar model architecture to Llama 2 [60] with the following details:

Table 11: Details of model architecture.

Hidden size	Context Len	Heads	Layers	Vocab size
1,024	2,048	16	24	32,000

Activation Function and Intermediate Hidden Size. We focus on dReLU and SwiGLU [52] activation functions.

Multi-Head Attention. For Attention block, we adopt the Llama-2-7B's architecture, apply pre-normalization using RMSNorm [66] and RoPE [57] for Positional embedding.

A.1.2 Training Hyperparameters

We utilize LLaMA-Factory as our training framework [70]. Our models are trained using AdamW optimizer [38], with the following hyper-parameters: $\beta_1 = 0.9, \beta_2 = 0.95$. We adopt a cosine learning rate schedule and we set weight decay to 0.01 and gradient clipping hyper-parameters. (see Table 12 for more details).

Table 12: Details of optimization hyper-parameters.

Sequence Length	Batch Size	Learning Rate	Warmup Steps	Hardware
2,048	128	$1e^{-4}$	2000	4 A100-80G GPUs

A.2 Activation Distribution Analysis of MoE Models

Figure 7 shows the activation distribution of Mistral and Mixtral. We can see that the FFN in MoE models show the similar activation distribution compared to dense Mistral models.

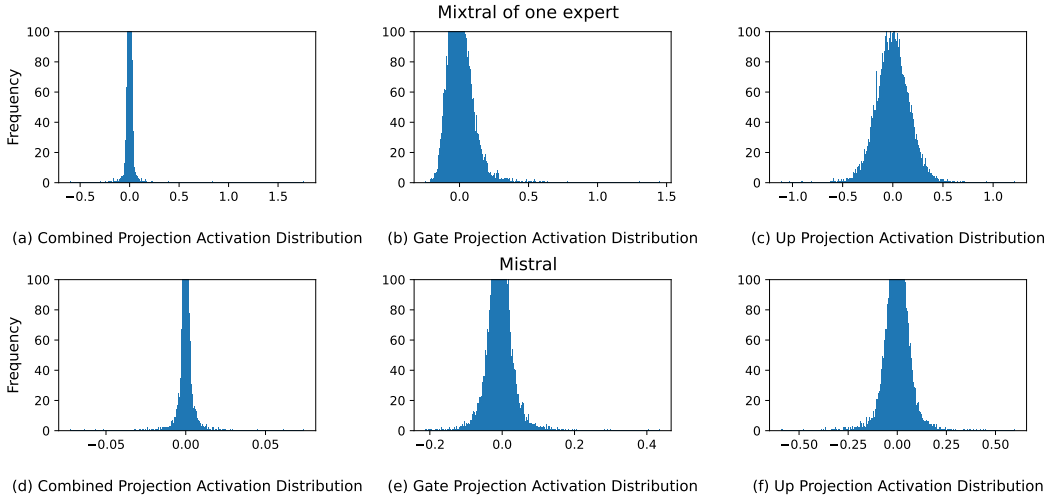


Figure 7: Post-activation distribution of Mixtral and Mistral.

A.3 Details Performance of ReLUfied Models

In this subsection, we present the detailed performance metrics of our ReLUfied models across various commonsense benchmarks, as shown in Table 13.

	Gemma -2B	Mistral -7B	TurboSparse -Mistral-7B	Mixtral	TurboSparse -Mixtral-47B
# Total Params	2B	7B	7B	47B	47B
# Activate Params	2B	7B	2.5B	13B	**4.3B**
SciQ	93.8	96.4	96.4	96.7	97.9
PIQA	76.71	80.79	80.58	82.43	82.15
OpenBookQA	39.8	46	47	49.4	49
ARC-Easy	74.12	80.3	81.06	83.75	85.06
All Avg.	71.11	75.87	76.26	78.07	78.53

Table 13: Common benchmarks results from four different models.

B Limitation

Our models have only undergone continued training on 150B tokens. Compared to the 15T tokens used in pre-training for Llama-3 [60], the limited number of training tokens still results in some deficiencies in the model's capabilities. We are optimistic that further training can help to mitigate these shortcomings.

C Broader Impact

The paper introduces a dReLU-based sparsification method and verifies its effectiveness on both dense and MoE LLMs. This approach significantly reduces computational demands, addresses environmental concerns through lower energy consumption, and helps democratize access to advanced AI technologies. We believe that our work can better support smaller organizations, educational institutions, and researchers, who previously faced barriers due to resource limitations, in accessing LLMs more easily.